

## **AGEX Development Plan**

### **1. 문서 개요**

#### **1.1 문서 목적**

본 문서는 AGEX Product Roadmap, Development Roadmap 및 WBS를 실제 개발 수행 체계로 전환하기 위한 공식 Development Plan이다.

본 문서는 다음 질문에 답한다.

- 어떤 개발 조직과 역할로 수행할 것인가.
- 어떤 기술 원칙을 적용할 것인가.
- 어떤 방식으로 Scope를 통제할 것인가.
- 어떤 순서로 개발·통합·검증할 것인가.
- 어떤 환경과 Toolchain을 사용할 것인가.
- 어떤 품질 기준으로 Release를 판단할 것인가.
- 어떤 위험을 어떻게 관리할 것인가.
- 어떤 산출물을 언제, 어떤 기준으로 완료할 것인가.

Development Roadmap이 개발의 "순서"를 정의하고 WBS가 "작업 구조"를 정의한다면, Development Plan은 실제 프로젝트를 어떻게 운영하고 구현할 것인지를 정의한다.

#### **1.2 적용 범위**

본 문서는 다음 영역을 포함한다.

- Development Governance
- Development Organization
- Technology Strategy
- Repository Strategy
- Architecture Implementation Strategy
- Environment Strategy
- API Development
- Runtime Development

- Agent/Workflow Development
- Knowledge/Memory Development
- Plugin/Model Development
- Security Development
- Multi-Tenant Development
- Governance Development
- DevOps
- QA
- AI Evaluation
- Release
- Documentation
- Risk
- Change Management
- Development Acceptance

### 1.3 Development Plan의 공식 정의

AGEX Development Plan은 AGEX의 공식 Architecture와 WBS를 실제 코드, Infrastructure, Test, Documentation 및 Release로 구현하기 위한 조직·기술·프로세스·품질·통제의 통합 실행 계획이다.

---

## 2. 개발 추진 기본 방향

AGEX 개발은 다음 기본 방향을 따른다.

1. Core Platform과 Runtime을 최우선으로 구축한다.
2. 특정 Application 기능은 Core에 포함하지 않는다.
3. API와 Schema를 코드보다 먼저 정의한다.
4. 초기에는 Modular Monolith를 우선한다.
5. 독립 Scaling이 필요한 영역만 별도 Service로 분리한다.

6. 모든 Model 호출은 Model Gateway를 거친다.
7. 모든 실행은 Runtime을 거친다.
8. Tenant Context는 전체 호출 경로에서 유지한다.
9. Security와 Observability는 기능 개발과 동시에 구현한다.
10. AI 품질은 Software Test와 별도의 Evaluation으로 검증한다.
11. Marketplace는 Plugin Contract가 안정된 이후 개발한다.
12. L4/L5 Autonomy는 Governance와 Runtime 안정화 이후 개발한다.

---

### 3. 개발 추진 구조

AGEX 개발은 다음 4개 축으로 운영한다.

Product / Architecture

|



Platform Engineering

|



AI / Runtime Engineering

|



Quality / Security / Operations

이를 실제 조직 관점에서 다음과 같이 구성한다.

- Product & Architecture
- Core Platform
- AI Runtime
- Knowledge & Memory

- Extension & Model
  - Frontend / Developer Experience
  - Platform / DevOps
  - Security
  - QA / Evaluation
- 

## **4. 개발 조직 기본 역할**

### **4.1 Product Owner / CPO**

주요 책임:

- Product Scope
- Priority
- Stage Gate
- Acceptance
- Roadmap
- Scope Change 승인

### **4.2 Chief AI Architect / Solution Architect**

주요 책임:

- Architecture Integrity
- Domain Boundary
- Runtime
- Agent
- Model Gateway
- AI Evaluation 기준
- Technical Decision Review

### **4.3 Platform Architect**

주요 책임:

- Core
- API
- Tenant
- Resource Model
- Event
- Data Architecture
- Deployment Pattern

#### **4.4 Principal Engineer / Tech Lead**

주요 책임:

- 실제 구현 기준
- Code Structure
- Review
- Technical Debt
- Performance
- Integration

#### **4.5 Security Architect**

주요 책임:

- Identity
- Authorization
- Tenant Isolation
- Secret
- Runtime Security
- Plugin Security
- Threat Modeling

## 4.6 DevOps / SRE

주요 책임:

- CI/CD
- Infrastructure
- Container
- Deployment
- Monitoring
- Backup
- Recovery

## 4.7 QA / AI Evaluation Lead

주요 책임:

- Test Strategy
- Integration
- E2E
- Security Test
- Performance
- Evaluation Dataset
- AI Regression

---

## 5. 개발팀 구성 원칙

초기 개발팀은 지나치게 Domain별로 분리하지 않는다.

권장 초기 Squad 구조:

### Squad A — Core Platform

담당:

- Tenant

- Resource Registry
- API
- Policy
- Audit
- Usage

### **Squad B — Runtime & AI**

담당:

- Runtime
- Agent
- Workflow
- Model Gateway

### **Squad C — Context & Extension**

담당:

- Knowledge
- Memory
- Plugin

### **Squad D — Platform Experience**

담당:

- SDK
- CLI
- Developer Console
- Documentation

### **Cross Functional**

- Security
- QA

- DevOps
  - Architecture
- 

## 6. 개발 단계 운영

전체 프로젝트는 다음 구현 Phase로 운영한다.

Phase 0 Development Foundation

Phase 1 Core Platform

Phase 2 Runtime

Phase 3 Agent / Workflow

Phase 4 Knowledge / Memory / Plugin

Phase 5 Enterprise Control

Phase 6 Marketplace

Phase 7 Autonomous Platform

각 Phase는 다음 내부 Cycle을 반복한다.

Design

→ Contract

→ Implement

→ Unit Test

→ Integration

→ Security

→ Evaluation

→ Acceptance

→ Release

---

## 7. 개발 방식



## 7.1 Iterative Development

대규모 기능 전체를 한 번에 구현하지 않는다.

각 Capability를 작은 실행 단위로 분리한다.

예:

Runtime:

Execution Resource

→ State Machine

→ Queue

→ Worker

→ Retry

→ Timeout

→ Checkpoint

→ Recovery

## 7.2 Vertical Slice

가능한 시점부터 End-to-End Slice를 구성한다.

예:

API

→ Execution

→ Worker

→ Model Gateway

→ Agent

→ Result

이를 통해 Architecture Integration을 조기에 검증한다.

---

## 8. 초기 End-to-End 목표

초기 개발의 첫 번째 중요한 목표는 복잡한 Workflow가 아니다.

다음 최소 흐름을 가능한 빠르게 완성해야 한다.

Tenant

→ Agent Definition

→ Publish

→ Execution Request

→ Runtime

→ Model Gateway

→ Model

→ Result

→ Audit

이 흐름이 안정화된 이후 Workflow, Knowledge, Memory 및 Plugin을 추가한다.

---

## 9. Repository 전략

초기에는 Monorepo를 권장한다.

이유:

- Contract 공유
- Schema 공유
- Atomic Change
- 개발 초기 Domain 이동
- SDK Generation
- 통합 테스트

예시:

agex/

└─ apps/

```
|   ├── api
|   ├── console
|   ├── worker
|   └── cli
|
├── packages/
|   ├── core
|   ├── runtime
|   ├── agent
|   ├── workflow
|   ├── knowledge
|   ├── memory
|   ├── plugin
|   ├── model
|   ├── security
|   └── sdk
|
├── schemas/
├── tests/
├── infrastructure/
└── docs/
```

---

## 10. Modular Monolith 전략

초기 Core는 논리적으로 분리된 Modular Monolith를 권장한다.

Domain 예:

Tenant Module

Resource Module

Policy Module

Runtime Module

Agent Module

Workflow Module

Knowledge Module

Memory Module

Plugin Module

Model Module

각 Module은 다음 원칙을 지킨다.

- 독립 Domain Model
- 독립 Repository Interface
- 명시적 Service Contract
- 직접 Cross-Domain DB 접근 금지

---

## 11. Service 분리 기준

다음 조건이 충족될 때 별도 Service로 분리한다.

- 독립 Scaling 필요
- 보안 경계 필요
- 다른 Runtime 필요
- 장애 격리 필요
- 별도 Deployment Cycle 필요
- 데이터 소유권이 명확히 분리됨

초기부터 Service 수를 늘리는 것을 목표로 하지 않는다.

---

## 12. 기술 Stack 선정 원칙

기술 Stack은 다음 기준으로 선정한다.

- 안정성
- 장기 유지보수
- 개발자 생태계
- Cloud Native 지원
- 성능
- Security
- 운영 복잡도
- Provider 독립성
- AI 생태계 호환성

---

## 13. Backend Language 전략

AGEX Core Backend는 가능한 한 하나의 Primary Language로 통일한다.

선정 시 고려:

- API 개발 생산성
- 동시성
- Worker
- Type Safety
- SDK 생태계
- AI Library 연동
- 유지보수성

AI-specific Worker는 필요할 경우 별도 Language Runtime을 허용할 수 있다.

단, Domain Contract는 언어에 독립적이어야 한다.

---

## 14. Frontend 전략

Developer Console은 Platform API Client로 동작한다.

원칙:

- Database 직접 접근 금지
- UI 전용 비공개 Core Logic 금지
- API와 동일 Permission 사용
- API와 동일 Lifecycle 사용

Frontend Framework는 Architecture Source of Truth가 아니다.

---

## 15. Database 개발 계획

초기 Primary Transaction Store는 관계형 Database를 권장한다.

저장 대상:

- Tenant
- Resource
- Version
- Policy
- Execution
- Workflow State
- Configuration
- Registry
- Governance

### 15.1 Repository Pattern

Domain이 SQL Schema에 직접 종속되지 않도록 Repository Interface를 둔다.

### 15.2 Migration

모든 Schema Change는 Migration으로 관리한다.

---

## 16. 데이터 저장소 분리 계획

AGEX는 데이터 특성에 따라 Storage를 분리한다.

### RDBMS

- Transactional State

### Object Storage

- Artifact
- File
- Package
- Large Input/Output

### Search

- Search Index

### Vector Store

- Knowledge/Memory Semantic Retrieval

### Cache

- Temporary State
- Rate Limit
- Cache

### Queue

- Task Dispatch
- 

## 17. API 개발 계획

API 개발은 다음 순서를 따른다.

Domain Requirement

- Resource Contract
- Schema
- Endpoint Contract
- Authorization
- Error Contract
- Mock
- Server
- Contract Test
- SDK

### **17.1 API First**

Frontend 개발 요구를 이유로 임시 Endpoint를 추가하지 않는다.

### **17.2 Version**

Breaking Change는 명시적 API Version 정책을 따른다.

---

## **18. Schema 개발 계획**

Canonical Schema는 다음에 사용한다.

- API
- SDK
- Event
- Plugin
- Resource
- Validation

Schema 변경 시 자동 Compatibility Check를 수행한다.

---

## **19. Runtime 개발 계획**



Runtime은 AGEX 개발의 가장 높은 우선순위를 가진다.

구현 순서:

Execution Resource

→ Execution State Machine

→ Task

→ Queue

→ Worker

→ Retry

→ Timeout

→ Cancellation

→ Checkpoint

→ Recovery

→ Fair Scheduling

## 19.1 첫 번째 성공 기준

Worker를 강제로 종료해도 실행이 복구되는지 검증한다.

---

## 20. Runtime 신뢰성 개발

다음 Failure Scenario를 초기부터 테스트한다.

- Worker Crash
- Queue Redelivery
- Network Timeout
- Model Timeout
- Database Retry
- Duplicate Task
- Cancellation

- Deployment During Execution

Runtime 안정성 검증 전 높은 Agent Autonomy를 개발하지 않는다.

---

## 21. Agent 개발 계획

Agent 개발 순서:

Agent Resource

→ Model Integration

→ Context Builder

→ Basic Agent Loop

→ Structured Output

→ Tool Call

→ Approval

→ Knowledge

→ Memory

→ Planning

→ Delegation

### 21.1 초기 Agent

초기에는 L0~L2에 집중한다.

---

## 22. Agent Context 개발

Context Builder는 독립 Component로 개발한다.

Input:

- Agent Specification
- Execution Context
- User Input

- Knowledge
- Memory
- Tool Result
- Policy

Output:

- Bounded Model Context

Context Window 초과 시 Priority와 Budget으로 제한한다.

---

## 23. Agent Action Validation

모델이 Tool Call을 생성한 후 즉시 실행하지 않는다.

표준 흐름:

Model Tool Call

→ Schema Validation

→ Permission

→ Policy

→ Side Effect

→ Approval

→ Runtime Action

---

## 24. Workflow 개발 계획

Workflow는 Agent 개발과 병행할 수 있으나 Runtime 위에서 구현한다.

초기:

- Sequential
- Condition
- Agent

- Model
- Function
- Wait
- Approval

후속:

- Parallel
- Join
- Loop
- Event
- Sub-Workflow
- Compensation

---

## 25. Workflow Editor 개발 계획

Visual Workflow Editor는 Engine Contract 안정화 이후 진행한다.

개발 순서:

Workflow Schema

→ Engine

→ API

→ Graph Renderer

→ Editor

Editor UI가 Workflow Definition을 결정하지 않는다.

---

## 26. Model Gateway 개발 계획

Model Gateway는 Agent Framework와 병행하여 초기부터 개발한다.

구현 순서:

1. Model Registry
  2. Provider Interface
  3. Provider Adapter
  4. Common Request
  5. Common Response
  6. Streaming
  7. Usage
  8. Health
  9. Basic Router
  10. Fallback
- 

## **27. Model Provider 통합 계획**

초기에는 제한된 수의 Provider를 선택한다.

목표는 Provider 수가 아니라 Adapter Contract 검증이다.

Direct Provider와 Private Provider 모두 동일 Interface로 연결 가능해야 한다.

---

## **28. Model Router 고도화 계획**

### **초기**

- Capability
- Availability
- Tenant Allowlist

### **Enterprise**

- Security
- Data Classification
- Region

- Cost
- Quality
- Latency

### **Advanced**

- Runtime Performance 기반 최적화 Candidate

---

## **29. Knowledge 개발 계획**

Knowledge는 다음 순서로 구축한다.

Source

→ Ingestion

→ Parser

→ Normalize

→ Metadata

→ Chunk

→ Embedding

→ Index

→ Retrieval

→ Citation

초기부터 Source Provenance를 유지한다.

---

## **30. Knowledge Connector 개발**

Connector Interface를 먼저 정의한다.

connect

healthCheck

list

fetch

metadata

changeDetection

구체 Source Connector는 후속 구현한다.

---

### 31. Knowledge Retrieval 품질 관리

Retrieval 품질은 별도 Evaluation Set으로 측정한다.

지표 예:

- Recall
  - Precision
  - Relevance
  - Citation Accuracy
  - Groundedness
- 

### 32. Memory 개발 계획

Memory는 초기부터 무제한 장기 저장 방식으로 구현하지 않는다.

순서:

Working

→ Session

→ Agent

→ User

→ Retrieval

→ Retention

→ Delete

→ Episodic

→ Semantic

→ Procedural

---

### 33. Memory Write 정책

모든 Candidate Memory를 저장하지 않는다.

다음은 평가한다.

- Scope
  - Relevance
  - Classification
  - Confidence
  - Retention
  - Duplicate
- 

### 34. Plugin 개발 계획

Plugin Framework 구현 순서:

Manifest

→ Registry

→ Installation

→ Configuration

→ Secret Binding

→ Action Contract

→ Plugin Gateway

→ Runtime

→ Sandbox

→ SDK



---

### 35. Plugin Sandbox 단계

#### 초기

Trusted Internal Adapter 중심.

#### 다음 단계

Isolated Container.

#### Enterprise

Risk-based Sandbox / Hardened Runtime.

Public Marketplace 이전에 Sandbox 구조를 검증한다.

---

### 36. Plugin Tool 관계 구현

안정적인 내부 계층은 다음을 사용한다.

Plugin Package

↓

Capability

↓

Action

↓

Agent-facing Tool Projection

Agent Tool과 Plugin Package를 동일 Resource로 취급하지 않는다.

---

### 37. Multi-Tenant 개발 계획

Tenant Isolation은 각 Workstream 완료 조건으로 포함한다.

검증 대상:

- API

- Database
- Cache
- Queue
- Event
- Search
- Vector
- Memory
- Runtime
- Plugin
- Secret

Cross-Tenant 테스트는 CI에 포함한다.

---

### **38. Security 개발 계획**

Security는 다음 4개 Layer로 병행 개발한다.

#### **Identity**

Authentication / Service Identity

#### **Authorization**

RBAC / Policy / ABAC

#### **Runtime Security**

Worker / Sandbox / Network

#### **AI Security**

Prompt Injection / Tool Injection / Data Exfiltration

---

### **39. Secret 관리 개발**

Secret은 전용 Interface를 먼저 만든다.

필요 기능:

- put
- reference
- resolve
- rotate
- revoke
- audit

Domain Resource는 Secret Value를 저장하지 않는다.

---

#### **40. Governance 개발 계획**

Governance는 Stage별로 확장한다.

##### **초기**

- Policy
- Basic Approval
- Audit

##### **중기**

- Risk
- Evaluation
- Resource Gate

##### **Enterprise**

- Exception
- Change Request
- Model Admission
- Plugin Admission
- Governance Dashboard

---

## 41. Observability 개발 계획

모든 Service와 Worker는 공통 Telemetry Library를 사용한다.

필수:

- Request ID
- Trace ID
- Execution ID
- Tenant Context
- Service
- Version
- Latency
- Error

민감 정보는 제외한다.

---

## 42. Developer Console 개발 계획

Console 우선순위:

### Step 1

- Tenant
- Agent
- Model
- Execution

### Step 2

- Workflow
- Knowledge
- Plugin

### Step 3

- Memory
- Audit
- Usage

### Step 4

- Governance
- Marketplace
- Enterprise Operations

---

## 43. SDK 개발 계획

SDK는 API Contract에서 생성 가능한 부분과 Hand-Written Layer를 분리한다.

Canonical Schema

→ Generated API Client

→ Core SDK

→ Agent / Workflow / Plugin SDK

초기 Tier 1 언어는 개발 역량과 사용자 생태계를 기준으로 제한한다.

---

## 44. CLI 개발 계획

초기 CLI는 개발자가 UI 없이 전체 핵심 흐름을 수행할 수 있어야 한다.

예:

agex auth login

agex tenant use

agex agent create

agex agent publish

agex agent run

agex execution get

후속:

agex workflow

agex plugin

agex knowledge

agex package

---

## 45. Local Runtime 계획

개발 생산성을 위해 Local Runtime을 제공한다.

초기 지원:

- Local Agent
- Mock Model
- Workflow Simulation
- Mock Plugin
- Local Trace

단, Local 성공이 Production 권한 승인을 의미하지 않는다.

---

## 46. DevOps 개발 계획

DevOps 구축 순서:

1. Local Container
2. CI
3. Development Deployment
4. Staging
5. Production
6. Autoscaling

- 7. Backup
  - 8. Restore
  - 9. Multi-Zone
  - 10. Private Deployment
- 

## **47. CI 구축**

기본 Pipeline:

Checkout

- Install
- Lint
- Unit Test
- Schema Test
- Contract Test
- Security Scan
- Build
- Image
- SBOM

Fail 발생 시 Release Artifact를 만들지 않는다.

---

## **48. CD 구축**

Signed Artifact

- Development
- Integration
- Staging
- E2E

→ Acceptance Gate

→ Production

Production은 Build를 다시 수행하지 않는다.

---

## 49. Infrastructure as Code

다음 영역을 IaC로 관리한다.

- Network
- Cluster
- Database
- Storage
- Queue
- IAM
- Monitoring

운영 환경 수동 설정을 Source of Truth로 사용하지 않는다.

---

## 50. Release 전략

Release 유형:

- Internal
- Alpha
- Private Preview
- Beta
- GA

각 Release마다 별도 Acceptance Gate를 가진다.

---

## 51. Release Candidate



GA 이전에는 Release Candidate를 생성한다.

RC 검증:

- Functional
  - Security
  - Tenant Isolation
  - Performance
  - Upgrade
  - Rollback
  - Documentation
- 

## 52. Feature Flag

위험도가 높은 신규 Runtime, Router 및 Planner는 Feature Flag를 통해 제한적으로 배포할 수 있다.

Feature Flag는 Core Security를 우회하는 용도로 사용할 수 없다.

---

## 53. Testing Strategy

전체 Test Pyramid:

E2E

Integration

Contract Test

Component Test

Unit Test

AI 기능은 별도 Evaluation Layer를 가진다.

---

## 54. Unit Test

대상:

- Domain Rule
- State Transition
- Validation
- Utility
- Policy Rule

주요 Core Logic에는 높은 수준의 Unit Coverage를 목표로 한다.

단, Coverage 숫자 자체를 품질 목표로 사용하지 않는다.

---

## 55. Contract Test

필수 대상:

- API
- Event
- Plugin
- Model Adapter
- Connector
- SDK

Contract가 변경되면 자동 Compatibility Test를 수행한다.

---

## 56. Integration Test

주요 조합:

- API + Core
- Runtime + Queue
- Agent + Model Gateway
- Workflow + Runtime

- Knowledge + Vector
  - Plugin + Secret Broker
- 

## 57. End-to-End Test

대표 E2E:

Tenant

→ Agent

→ Knowledge

→ Workflow

→ Plugin

→ Approval

→ Result

→ Audit

---

## 58. Failure Test

Runtime 핵심 기능에는 장애 테스트를 의무화한다.

- Kill Worker
  - Duplicate Queue
  - Model Timeout
  - Plugin Timeout
  - DB Temporary Failure
  - Cancellation
  - Deployment During Execution
- 

## 59. Security Test

필수:

- Authentication Bypass
  - Object Authorization
  - Cross-Tenant
  - Prompt Injection
  - Tool Injection
  - Secret Leakage
  - SSRF
  - Plugin Sandbox
  - Replay
- 

## 60. AI Evaluation Plan

AI 기능에는 Software Test와 별도로 Evaluation을 수행한다.

평가 영역:

- Task Success
  - Groundedness
  - Tool Selection
  - Structured Output
  - Retrieval
  - Policy Compliance
  - Cost
  - Latency
- 

## 61. Evaluation Dataset

Evaluation Dataset은 공식 Resource처럼 관리한다.

필드:

- Dataset ID
- Owner
- Version
- Scope
- Expected Outcome
- Risk Category

운영 데이터만으로 평가 기준을 만들지 않는다.

---

## 62. Regression Evaluation

Agent, Model, Prompt Policy, Retrieval 또는 Planner 변경 시 이전 Version과 비교한다.

결과 예:

Baseline Success: 87%

Candidate Success: 83%

→ Promotion Block

정확한 Threshold는 Resource별로 정의한다.

---

## 63. Performance Test

주요 지표:

- API Latency
- Execution Start Latency
- Queue Wait
- Agent Duration
- Model Gateway Overhead

- Retrieval Latency
  - Plugin Latency
- 

## 64. Load Test

다음 부하를 별도로 검증한다.

- API Burst
  - Agent Concurrent Execution
  - Workflow Long Running
  - Knowledge Bulk Ingestion
  - Model Streaming
  - Plugin Burst
- 

## 65. Chaos Test

Enterprise 단계부터 다음을 포함한다.

- Worker Kill
  - Node Failure
  - Queue Delay
  - Database Failover
  - Model Provider Down
  - Network Partition
- 

## 66. Quality Gate

Feature 완료 조건:

Code

+ Test

- + Security
- + Observability
- + Documentation
- + Acceptance

이 중 하나라도 누락되면 Production Ready가 아니다.

---

## 67. 개발 산출물

각 주요 기능은 최소 다음 산출물을 가진다.

- Source Code
- Schema
- API Contract
- Migration
- Test
- Telemetry
- Documentation
- Release Note

고위험 기능은 추가로:

- Threat Model
  - Security Review
  - Failure Test
  - Runbook
- 

## 68. Architecture Decision Record

주요 기술 결정은 ADR로 기록한다.

ADR 필드:

- Decision
- Context
- Options
- Selected
- Reason
- Consequence
- Date
- Owner

---

## 69. 개발 문서 관리

Architecture 문서와 Code의 불일치를 방지한다.

변경 유형:

### 구현 세부 변경

ADR로 관리.

### Architecture 변경

해당 공식 AGEX Architecture 문서를 수정해야 한다.

---

## 70. Scope 관리

AGEX Core에 기능을 추가하기 전 다음을 확인한다.

General AI OS Capability?

|

|— Yes → Core 후보

|

|— No

↓



Application-specific?

|

|— Yes → Core 제외

|— No → Architecture Review

---

## 71. Scope Creep 방지

다음 항목은 Core Scope로 쉽게 추가하지 않는다.

- 특정 산업 기능
- 특정 업무 승인 프로세스
- 범용 Office 기능
- 특정 고객 UI
- 특정 Provider 전용 기능

---

## 72. 변경 요청 관리

중요 Scope 변경은 다음을 기록한다.

- Change ID
- Reason
- Value
- Architecture Impact
- Schedule Impact
- Resource Impact
- Cost Impact
- Priority

---

## 73. 개발 우선순위

Priority:

- P0 Architecture Critical
- P1 Product Critical
- P2 Enterprise / Scale
- P3 Ecosystem / Optimization

P0 미완료 상태에서 P3 개발에 핵심 인력을 전환하지 않는다.

---

#### 74. 개발 일정 산정 원칙

상세 기간은 WBS Estimate를 기반으로 산정한다.

기간 계산 시 다음을 포함해야 한다.

- Design
- Implementation
- Test
- Integration
- Security
- Documentation
- Stabilization

Coding Time만 일정으로 계산하지 않는다.

---

#### 75. 개발 추정 단위

WBS Task는 다음 기준으로 추정할 수 있다.

- Story Point
- Engineer-Day
- Engineer-Week

Cost Estimation과 Resource Plan에서는 Engineer-Month로 집계할 수 있다.

---

## 76. 공수 산정 분리

다음 공수는 별도로 식별한다.

- Architecture
- Backend
- Frontend
- AI
- DevOps
- Security
- QA
- Documentation
- Management

QA와 DevOps를 Backend 공수에 숨기지 않는다.

---

## 77. 개발 생산성 가정

AGEX는 Architecture 밀도가 높은 Platform이므로 일반 CRUD 시스템보다 다음 영역에 더 많은 공수가 필요하다.

- Contract Design
- Runtime
- Failure Handling
- Multi-Tenant
- Security
- Testing
- AI Evaluation

기능 화면 수만으로 공수를 산정하지 않는다.

---

## 78. AI Coding Agent 활용 원칙

AI Coding Agent를 개발 생산성 향상 도구로 사용할 수 있다.

활용 가능:

- Boilerplate
- Unit Test 초안
- Refactoring
- Schema Client Generation
- Documentation 초안
- Static Analysis 보조

그러나 다음은 Human Review 없이 승인하지 않는다.

- Security Policy
- Authorization
- Tenant Isolation
- Runtime State
- Database Migration
- Secret Handling
- Production Infrastructure

---

## 79. AI 생성 코드 검증

AI 생성 여부와 관계없이 동일 품질 기준을 적용한다.

필수:

- Code Review
- Test
- Security Scan

- Contract Validation

AI가 작성했다는 이유로 별도 낮은 기준을 적용하지 않는다.

---

## 80. 개발 환경

개발자는 가능한 동일 Container 기반 환경을 사용한다.

최소 구성:

- Backend
- Database
- Queue
- Cache
- Object Storage
- Mock Model

이를 통해 개인별 환경 차이를 최소화한다.

---

## 81. Development 환경

공유 Development 환경은 다음 목적에 사용한다.

- Service Integration
- API Test
- Shared Demo
- CI Deployment

개발자 개인 기능 테스트는 Local 환경을 우선한다.

---

## 82. Integration 환경

복수 Workstream 간 Contract를 검증한다.

예:

- Runtime + Agent
  - Workflow + Plugin
  - Knowledge + Agent
  - SDK + API
- 

### 83. Staging 환경

Production과 가능한 유사하게 구성한다.

검증:

- Deployment
  - Migration
  - Security
  - Performance
  - Upgrade
  - Rollback
- 

### 84. Production 환경

Production 변경은 CI/CD Pipeline과 승인 절차를 사용한다.

운영 서버에서 Source Code를 직접 수정하지 않는다.

---

### 85. Database Migration 운영

Migration은 다음 유형으로 구분한다.

- Backward Compatible
- Data Backfill
- Breaking

Breaking Migration은 다단계로 수행한다.

Expand

→ Deploy

→ Backfill

→ Verify

→ Contract

---

## 86. Runtime Compatibility 관리

장기 실행 중 Platform Upgrade가 발생할 수 있으므로 다음을 기록한다.

- Runtime Version
- Worker Version
- Resource Version
- Policy Version

---

## 87. Backward Compatibility 정책

기존 다음 Resource가 신규 Release에서도 동작하는지 검증한다.

- Agent
- Workflow
- Plugin
- API Client
- SDK
- Long Running Execution

---

## 88. Deprecation 개발 절차

기능 제거 순서:

Mark Deprecated

- Warn
  - Provide Replacement
  - Migration
  - End Support
  - Remove
- 

## 89. Technical Debt 관리

Debt 항목은 Backlog에 공식 등록한다.

필드:

- Debt ID
  - Cause
  - Impact
  - Risk
  - Owner
  - Removal Target
- 

## 90. Release Management

각 Release는 Release Manifest를 가진다.

포함:

- Platform Version
- Service Versions
- API Version
- Schema Version
- Runtime Version
- Migration Version



- Known Issues

---

## 91. Release Gate

Production Release 전 다음을 확인한다.

1. Build 성공
2. Unit Test
3. Contract Test
4. Integration Test
5. Tenant Isolation
6. Security Scan
7. Migration Test
8. Observability
9. Documentation
10. Rollback

---

## 92. Rollback 계획

Rollback은 Release 설계 단계에서 정의한다.

확인:

- Application Rollback
- Database Compatibility
- Config Compatibility
- Runtime Compatibility
- Event Compatibility

---

## 93. Incident 대응 준비

GA 전 주요 Service에 Runbook을 작성한다.

대표:

- API 장애
- DB 장애
- Queue Backlog
- Runtime Worker 장애
- Model Gateway 장애
- Plugin 장애
- Security Incident

---

#### 94. 개발 현황 보고 기준

프로젝트 상태는 단순 전체 퍼센트 하나만 사용하지 않는다.

최소 다음으로 구분한다.

- Architecture Completion
- Implementation Completion
- Test Completion
- Integration Completion
- Security Completion
- Release Readiness

---

#### 95. Progress 계산

권장 계산 방식:

Progress =

Design

+ Implementation

+ Test

+ Integration

+ Acceptance

각 영역별 Weight는 Workstream 성격에 따라 설정한다.

Code가 완료되었다고 100%로 계산하지 않는다.

---

## 96. Milestone 보고

주요 보고 Milestone:

- M0 Development Ready
- M1 Core Ready
- M2 Runtime Ready
- M3 Agent MVP
- M4 Workflow MVP
- M5 Context Platform
- M6 Extension Platform
- M7 Enterprise Ready
- M8 Ecosystem Ready
- M9 Autonomous Platform

---

## 97. 개발 리스크 관리

주요 Risk:

- Architecture 변경
- Runtime 복잡도
- Tenant Isolation
- Model Provider 변화

- Plugin Security
  - AI Evaluation 불확실성
  - Technical Debt
  - Scope Expansion
  - Infrastructure Cost
- 

## 98. Risk 대응 구조

각 Risk는 다음 정보를 가진다.

- Risk ID
  - Probability
  - Impact
  - Severity
  - Owner
  - Mitigation
  - Trigger
  - Contingency
- 

## 99. Critical Risk 1 — Runtime

위험:

장기 실행과 장애 복구가 안정적으로 동작하지 않음.

대응:

- Runtime 우선 개발
- Failure Injection
- Durable State
- Checkpoint

- Idempotency

---

## 100. Critical Risk 2 — Tenant Isolation

위험:

Cross-Tenant 데이터 노출.

대응:

- Tenant Context Everywhere
- Repository Enforcement
- Search/Vector Isolation
- Automated Cross-Tenant Test

---

## 101. Critical Risk 3 — Provider 종속

위험:

Agent와 Application이 특정 Provider API에 결합.

대응:

- Model Gateway
- Provider Interface
- Common Model Contract
- Adapter Test

---

## 102. Critical Risk 4 — Plugin

위험:

외부 Plugin이 Core 또는 Tenant Data를 침해.

대응:

- Sandbox

- Manifest Permission
  - Network Control
  - Secret Broker
  - Security Scan
- 

### 103. Critical Risk 5 — AI 품질

위험:

Software는 정상이나 AI 결과 품질이 불안정.

대응:

- Evaluation Dataset
  - Regression
  - Model Evaluation
  - Retrieval Evaluation
  - Quality Gate
- 

### 104. Critical Risk 6 — Scope

위험:

AGEX Core가 특정 Application 요구로 비대화.

대응:

- Product Boundary
  - Architecture Review
  - Application Layer 분리
- 

### 105. 문서화 계획

다음 공식 문서를 코드와 Release에 맞춰 유지한다.

- Product Documentation
- API Guide
- Administrator Guide
- Developer Guide
- SDK Reference
- Operations Guide

Architecture 문서는 구현 변경에 따라 Revision 관리한다.

---

## 106. Developer Guide 동기화

새 Public 기능은 완료 Definition에 Developer Guide Update를 포함한다.

예:

Plugin API 변경 시:

Code

+ API

+ SDK

+ Plugin Guide

---

## 107. 운영 인수 준비

GA 전 개발팀에서 운영팀으로 다음을 인계한다.

- Architecture
- Deployment
- Monitoring
- Alert
- Runbook
- Backup

- Recovery
  - Known Issues
  - Escalation
- 

## 108. Security 인수 준비

다음 정보를 Security 운영으로 인계한다.

- Threat Model
  - Security Baseline
  - High-Risk Resources
  - Kill Switch
  - Audit
  - Incident Procedure
  - Vulnerability Process
- 

## 109. MVP 개발 범위

첫 MVP에는 다음을 포함한다.

### 반드시 포함

- Tenant
- Resource Registry
- Schema
- API
- Runtime
- Model Gateway
- Agent
- Basic Workflow



- Basic Knowledge
- Basic Memory
- Plugin Core
- Authentication
- Authorization
- Audit
- Usage
- Observability
- SDK/CLI 기본

#### 제외 가능

- Public Marketplace
- Full Commercial Billing
- L4/L5
- Federation
- Advanced Graph Knowledge
- Full On-Prem Automation

---

### 110. MVP 개발 성공 시나리오

MVP 완료 판단을 위한 대표 Scenario:

1. Tenant 생성
2. 사용자 인증
3. Model Profile 생성
4. Knowledge Source 등록
5. Plugin 설치
6. Agent 생성

7. Workflow 생성
8. Execution 시작
9. Knowledge Retrieval
10. Agent Model 호출
11. Plugin Action 요청
12. Approval
13. Plugin 실행
14. 결과 생성
15. Memory Proposal
16. Execution 완료
17. Audit 조회
18. Usage 조회

이 Scenario는 UI 없이 API/CLI만으로도 수행할 수 있어야 한다.

---

## **111. MVP Technical Acceptance**

다음 조건을 충족해야 한다.

- Worker Failure Recovery
- Tenant Isolation
- Agent Version Pinning
- Workflow Resume
- Plugin Secret Protection
- Model Provider Abstraction
- Knowledge Citation
- Memory Delete
- Trace

- Audit
  - API/SDK Contract 일치
- 

### 112. Enterprise 개발 진입 조건

Enterprise Phase로 진입하기 전에 다음이 안정화되어야 한다.

- Runtime
  - Agent
  - Workflow
  - Plugin Contract
  - Tenant Isolation
  - Model Gateway
  - Observability
- 

### 113. Marketplace 개발 진입 조건

Public Marketplace는 다음 조건 이후 시작한다.

1. Plugin Manifest 안정화
  2. Package Versioning 안정화
  3. Sandbox 운영 가능
  4. Signature 검증 가능
  5. Upgrade/Rollback 가능
  6. Tenant Installation 안정화
  7. Security Admission 체계 확보
- 

### 114. L4 개발 진입 조건

L4 Continuous Agent 개발 전:

- Kill Switch
- Budget
- Risk
- Policy
- Audit
- Long Running Runtime
- Evaluation

이 모두 운영 수준으로 준비되어야 한다.

---

## 115. L5 개발 진입 조건

L5 개발 전:

- Multi-Agent Permission Model
- Delegation Depth
- Agent Identity
- Team Cost
- Collaboration Audit
- Dynamic Workflow Guard
- Governance

가 준비되어야 한다.

---

## 116. 개발 승인 체계

주요 변경은 다음과 같이 승인한다.

### Architecture

Architecture Review

### Public API

API Contract Review

## **Security**

Security Review

## **Database Breaking Change**

Architecture + Tech Lead Review

## **Production Release**

Release Gate

## **High-Risk AI Capability**

Governance Review

---

### **117. Code Review 기준**

Review 항목:

- Correctness
- Domain Boundary
- Tenant Context
- Security
- Error Handling
- Observability
- Test
- Performance
- Maintainability

단순 Style Review에 그치지 않는다.

---

### **118. PR 완료 기준**

PR Merge 전:

- CI Pass
  - Required Review
  - Test
  - No New Critical Security Finding
  - Schema Compatibility
  - Documentation where required
- 

## 119. 개발 속도와 Architecture 균형

초기에는 빠른 기능 구현보다 다음을 우선한다.

Correct Contract

>

Fast Feature

Durable Runtime

>

Demo Convenience

Tenant Isolation

>

Development Shortcut

반면 검증되지 않은 추상화에 과도한 시간을 쓰지 않는다.

실제 Vertical Slice를 통해 Architecture를 검증한다.

---

## 120. Prototype 관리

실험용 Prototype은 Production Source와 명확히 구분한다.

Prototype 목적:

- Technical Feasibility
- Model Test
- Performance
- UX Concept

Prototype 결과를 Production Architecture로 자동 승격하지 않는다.

---

## 121. Spike Task

불확실성이 높은 기술은 Time-boxed Spike를 사용한다.

예:

- Sandbox Technology
- Vector Store
- GPU Runtime
- Event Infrastructure

Spike 결과:

- Findings
  - Benchmark
  - Recommendation
  - ADR
- 

## 122. Build vs Buy 결정

다음 기준으로 외부 제품 사용 여부를 결정한다.

- AGEX Core Differentiator인가.
- 장기 Lock-in 위험은 어떤가.
- 운영 비용은 어떤가.

- Security가 충분한가.
- Adapter로 교체 가능한가.

Infrastructure 기능은 가능한 검증된 외부 기술을 활용한다.

AGEX 고유 Runtime, Resource, Agent 및 Governance Contract는 Core 역량으로 유지한다.

---

### 123. External Dependency 관리

모든 주요 외부 Dependency는 다음을 기록한다.

- Name
  - Version
  - License
  - Owner
  - Usage
  - Security Status
  - Replacement Option
- 

### 124. 오픈소스 활용 원칙

Open Source는 적극 활용할 수 있다.

단:

- License
- Maintenance
- Security
- Community
- Upgrade Path

를 검증한다.



Core Business Logic을 유지보수 중단 Project에 과도하게 종속하지 않는다.

---

### 125. Performance Budget

주요 Platform 기능에는 Performance Budget을 설정할 수 있다.

예:

- API Gateway Overhead
- Policy Evaluation
- Model Gateway Overhead
- Retrieval
- Scheduling

정확한 수치는 성능 측정 후 SLO에서 확정한다.

---

### 126. Cost Budget

개발 환경부터 다음 비용을 추적한다.

- Model API
- GPU
- Storage
- Logging
- Search
- Vector

개발·QA 환경의 AI 비용도 관리 대상이다.

---

### 127. Logging 비용 관리

AI Runtime에서는 Log 양이 급증할 수 있으므로 다음을 적용한다.

- Structured Log

- Sampling
- Retention Tier
- Payload Exclusion

Raw Prompt 전체를 기본 Log에 저장하지 않는다.

---

## 128. 팀 운영 리듬

권장 운영:

- Daily Engineering Sync
- Weekly Architecture Review
- Weekly Risk Review
- Sprint/Iteration Review
- Milestone Acceptance Review

구체적인 Scrum 방식 자체를 Architecture가 강제하지는 않는다.

---

## 129. Architecture Review Agenda

검토 항목:

- Domain Boundary
- Contract
- Data Ownership
- Tenant
- Security
- Failure
- Observability
- Deployment
- Backward Compatibility

---

### 130. 기술 부채 Review

정기적으로 다음을 확인한다.

- Temporary Code
- Duplicate Contract
- Deprecated API
- Hard-coded Provider
- Performance Bottleneck
- Weak Test

P0/P1 Debt는 제품 기능보다 우선할 수 있다.

---

### 131. 개발 품질 KPI

예:

- CI Pass Rate
- Change Failure Rate
- Deployment Success
- Escaped Defect
- Mean Recovery Time
- Contract Regression
- Security Finding

AI KPI:

- Task Success
- Groundedness
- Tool Correctness
- Cost per Successful Execution

---

### 132. 프로젝트 완료의 정의

AGEX 개발은 특정 Version 출시로 전체 프로젝트가 종료되는 형태가 아니다.

각 Stage는 독립적인 Product Maturity Level을 가진다.

예:

Stage 1 GA

= Core AI OS 확보

Stage 3 GA

= Enterprise AI OS 확보

Stage 4 GA

= Ecosystem Platform 확보

---

### 133. Development Plan Acceptance Criteria

본 Development Plan은 다음을 충족해야 한다.

1. Product Roadmap과 WBS를 실제 수행 방식으로 연결한다.
2. 개발 조직 역할을 정의한다.
3. Core와 Runtime 우선순위를 명확히 한다.
4. Modular Monolith 초기 전략을 정의한다.
5. Service 분리 기준을 정의한다.
6. API First와 Schema First를 적용한다.
7. Model Gateway를 강제한다.
8. Tenant Isolation을 모든 Workstream에 적용한다.
9. Security를 초기부터 포함한다.

10. Observability를 기능 완료 조건으로 포함한다.
  11. Software Test와 AI Evaluation을 분리한다.
  12. CI/CD와 Environment 전략을 정의한다.
  13. Database Migration 전략을 정의한다.
  14. Release Gate와 Rollback을 정의한다.
  15. MVP 범위와 Acceptance Scenario를 정의한다.
  16. Enterprise 진입 조건을 정의한다.
  17. Marketplace 진입 조건을 정의한다.
  18. L4/L5 진입 조건을 정의한다.
  19. Scope 변경과 Risk 관리 방식을 정의한다.
  20. Resource Plan 및 Cost Estimation으로 연결 가능하다.
- 

#### **134. Development Plan 금지사항**

다음 개발 방식은 허용하지 않는다.

- Architecture Contract 없이 바로 구현
- 특정 Application 요구를 Core 기능으로 바로 추가
- Agent에서 Model Provider 직접 호출
- Runtime 없이 장기 AI Task 구현
- Tenant Filter를 개발자 선택 사항으로 구현
- Worker Memory에 실행 상태만 저장
- Plugin에 장기 Secret 전달
- Public API와 Console Backend의 핵심 기능 분리
- Security를 Release 직전에 추가
- Observability 없는 Service 배포
- Test 없는 Code Merge를 완료 처리

- AI 품질을 Demo 성공만으로 판단
- Marketplace를 Plugin Contract 전에 구축
- L4/L5를 Governance 이전에 개발
- Production Environment에서 직접 Source 수정
- 동일 Version Artifact를 환경별로 재빌드
- Database Breaking Migration을 즉시 적용
- Rollback Plan 없는 Release
- Scope 변경의 비용·일정 영향 미평가
- AI Coding Agent 결과를 무검증 Production 적용
- Documentation을 최종 단계로 미룸

---

### 135. Development Plan의 최종 정의

AGEX Development Plan은 AGEX Architecture와 WBS를 실제 개발 프로젝트로 수행하기 위한 조직, 기술, 개발 방식, 품질, 보안, 테스트, 배포 및 운영 준비의 통합 기준이다.

AGEX 개발은 Agent 기능을 빠르게 보여주는 Demo 중심으로 시작하지 않는다.

먼저 Tenant, Resource Contract, Schema, API 및 Durable Runtime을 구축하고, 그 위에 Model Gateway와 Agent, Workflow를 구현한다.

Knowledge, Memory 및 Plugin은 Agent와 Workflow의 표준 Context 및 Extension Layer로 연결하며, 외부 기능과 Model Provider는 반드시 Adapter와 Gateway를 통해 추상화한다.

초기 Architecture는 Modular Monolith를 우선하여 Domain Boundary를 코드 수준에서 강제하고, 실제 독립 확장·보안·장애 격리 요구가 발생하는 영역만 단계적으로 Service로 분리한다.

Security, Multi-Tenant, Observability, QA 및 Documentation은 후속 단계가 아니라 모든 기능의 Definition of Done에 포함한다.

AI 기능은 일반 Software Test만으로 검증하지 않고 별도의 Evaluation Dataset, Regression 및 Quality Gate를 적용한다.

Production Release는 동일한 Immutable Artifact를 Development, Staging, Production으

로 Promotion하며, Migration, Compatibility, Rollback 및 Operational Readiness를 함께 검증한다.

Marketplace는 Extension Contract와 Sandbox가 안정화된 이후 개발하고, 높은 수준의 L4/L5 자율성은 Runtime-Governance-Security-Evaluation이 충분히 검증된 이후에만 확대한다.

AGEX Development Plan은 다음 문장으로 최종 정의한다.

**AGEX Development Plan은 범용 AI Operating System을 빠르게 만드는 계획이 아니라, 올바른 Resource Contract와 Durable Runtime을 기반으로 모든 AI 기능을 보안·Tenant·품질·운영 통제 안에서 단계적으로 구현하기 위한 AGEX의 공식 개발 수행 기준이다.**

본 문서는 AGEX 개발의 조직, 기술 전략, 구현 순서, 환경, 품질, Release 및 Risk 관리 방식을 규정하는 공식 Development Plan 문서로 사용한다.